

Video Game Programming: Mobile Game Programming

L16

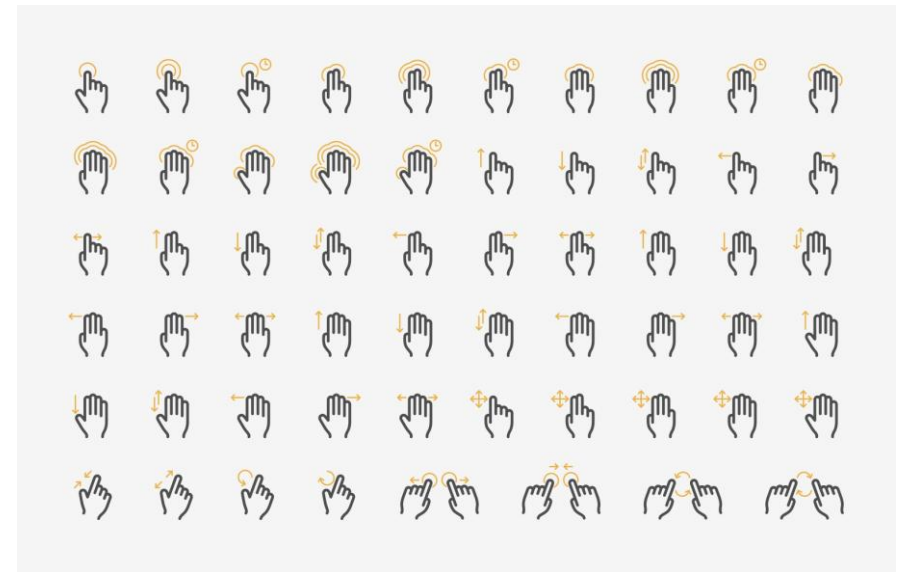
Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Mobile input

- Input on mobile is much different than on other platforms
- The main input device is the **touchscreen**
- Most mobile devices also support accelerometer, GPS, and camera
- Some also have gyroscope and compass
- Additional input devices can be connected using Bluetooth
- Porting a game from another platform to mobile will at least require reconsidering how players provide input
- Game engines such as Unity can help by simulating different mouse buttons with multitouch

Touch

- Touch enables direct and absolute pointing but not hovering
- Direct and absolute pointing
 - reduces undesired rotations
 - supports intuitive manipulation
 - has occlusion issues
- Players can interact using one to many fingers (multitouch)
- Multitouch allows players to perform gestures



Multitouch gesture icons by MediaLoot

Virtual joysticks

- An intuitive way to port joystick-based movement is using virtual joysticks
- One (usually on the left) is for movement and another one (right) for (fire) direction
- Joystick can be static or dynamic, always visible or displayed on touch, active or not outside the joystick area
- Fullscreen sliding can be used for rotation to replace the second joystick



Minigore and Geometry Wars virtual joysticks

Tap to move

- An interesting alternative to virtual joysticks is tap to move
- It builds over PC point-and-click technique
- Movement can be continuous or instantaneous (teleport)
- Rotation can be fixed, automatic, or controlled by a virtual joystick, by sliding in a specific area, or by the gyroscope



Tap to move / teleport
in Bastion and HCI Lab test

Touch detection in Unity

- Unity can detect touch events:
 - using EventSystems interfaces (e.g., IDragHandler)
 - using Input static properties and methods (e.g., touchCount, GetTouch)
 - using actions or EnhancedTouch of the new Input System
- For each finger touching the screen, the Touch class contains information such as position and phase (Began, Moved,...)
- Touch also counts taps, but a more general implementation can rely on Began / Ended phases at the same position in a time frame
- Unity Remote app can be used for debugging (now also with the new Input System)

Accelerometer

- Accelerometer reports linear acceleration along the three axes
- This data can be used to detect device orientation and its changes
- In Unity, last acceleration data is provided by `Input.acceleration` property (or by `Accelerometer.current.acceleration.ReadValue` in the new Input System) as multiples of g force, while `Input.deviceOrientation` provides a discrete evaluation of device orientation
- Accelerometer can provide more than one event per frame (in Unity, they are stored in `Input.accelerationEvents`)



Rolando and Real Racing series use accelerometer as control

Gyroscope

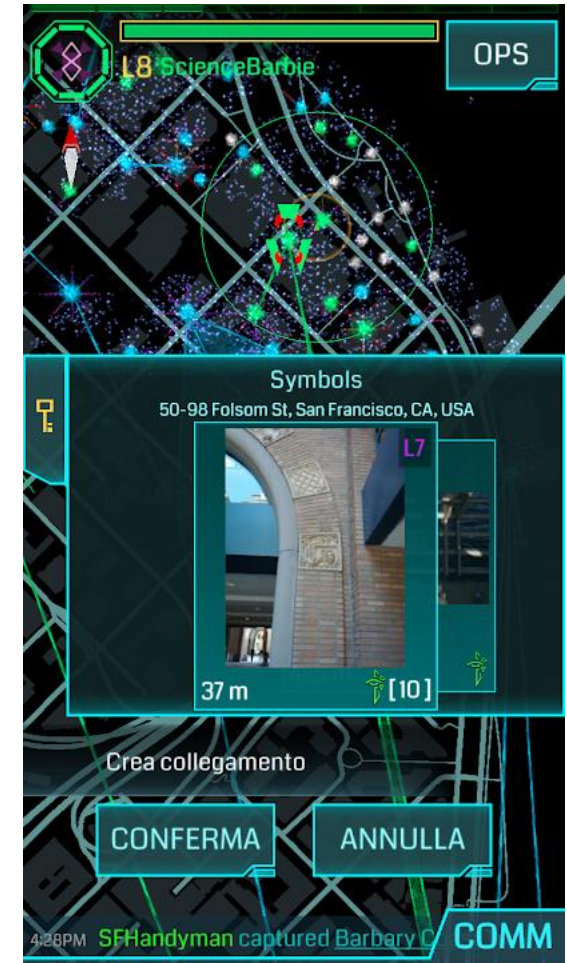
- Some devices embed a gyroscope
- Gyroscope data can be useful to rotate camera when players rotate the device
- It is easier to detect rotations with gyroscope than accelerometer
- In Unity, `Input.gyro` provides access to Gyroscope class (which uses more sensors)
- Gyroscope is right-handed and Unity left-handed, so a transformation is needed, e.g. `GyroToUnity(Input.gyro.attitude)` that returns a `Quaternion(q.x, q.y, -q.z, -q.w)`
- The new Unity Input System replaces gyro with a set of classes for different sensors



Gyroscope use in
Pokemon GO

Compass and GPS

- Compass and GPS provide geomagnetic orientation and global positioning data
- Mobiles OSs require user permission to use that data, so requests and players' responses should be handled
- Since these devices consume battery, game engines offer methods to start and stop use of these sensors
- Unity provides access to compass and GPS data with `Input.compass` and `Input.location`



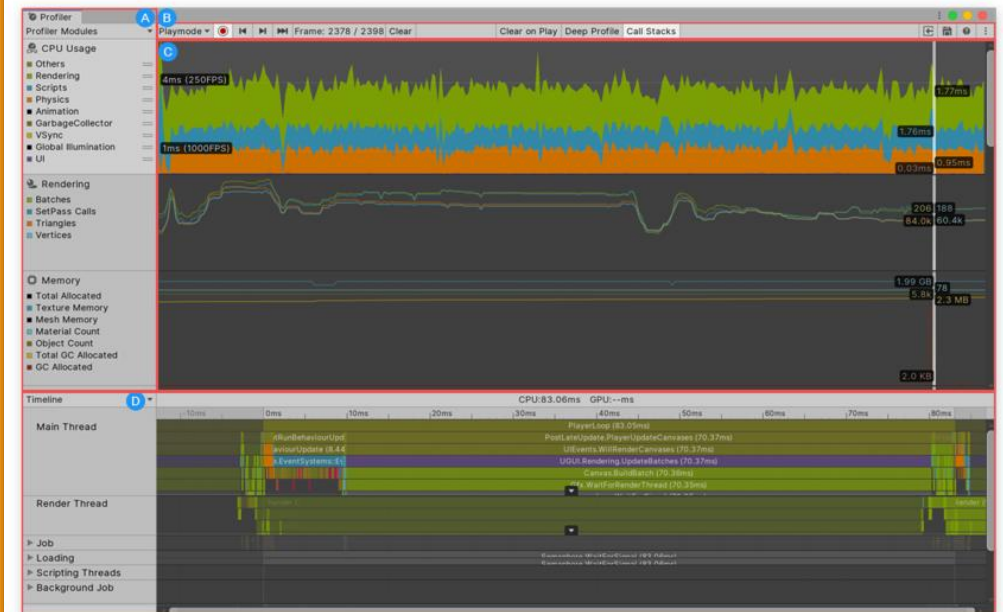
Ingress game using GPS

Mobile GUI

- Different smartphones and tablets have different **aspect ratios** (e.g., 16:9, 18:9, 3:2, 4:3)
- Different devices also have different **pixel densities**
- Mobile games thus need **responsive GUIs**
- Button placement should consider how a device is handheld
- Mobile OSs and stores can require icons and game screenshots for different densities and aspect ratios
- Game engine can provide GUI tools and help in setting some of the required resources

Profiling and optimization

- Performance of mobile devices greatly varies
- Performance check should be performed on several real devices (not in the game engine editor)
- Game engines offer profiling tools to check CPU, GPU, memory,...
- When a performance issue is identified, specific optimization is needed



Unity profiler

Asset optimization

- Assets suited for PC or console may not be suited for mobile
 - High poly 3D models may need to be replaced by low poly ones
 - Images should be compressed in a supported format
 - Image size may be decreased (replacement or override setting)
 - Audio quality may be reduced
- Supported formats vary on different devices, leading some developers to create specific builds
- Unnecessary read/write access should be disabled
- In Unity, unnecessary assets should be removed from Resources
- Runtime assets (e.g., asset bundles) should be used correctly

Graphics optimization

- Some settings should be tuned considering device performance (e.g., using the adaptive performance package in Unity):
 - Changing resolution
 - Enabling or disabling antialiasing and post-effects
 - Tuning amount of particles
 - Setting LOD bias...
- In general, limiting materials, textures (e.g., using atlases), transparency, runtime lights, and shadows is helpful
- Baking should be preferred over runtime light simulation
- Optimized pipelines can be used (e.g., Unity Universal RP)

Additional optimization

- Some additional optimization can be implemented in the editor or the code:
 - Marking non-moving objects as static
 - Limiting rigid bodies
 - Using primitive instead of mesh colliders
 - Avoiding moving static colliders
 - Limiting or disabling animated characters
 - Reusing instead of creating/destroying
 - Loading an empty scene between two complex scenes
 - ...

Build and deployment process

- To make gameplay and performance tests on real devices, the game must be built and deployed
- Android and iOS have specific build settings and processes
- All mobile platforms support a two-phase build: one in the game engine and the other with specific editors and SDKs
- Unity can handle Android build and deployment in a single phase without opening a specific editor but directly using the SDK

Bibliografy

- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th ed., Packt Publishing, 2024

